# Web3.0 Unveiled - Day 1

Mentored by FEC

## How to get started with programming?

To get started as a Web3 developer, we need some background knowledge of Web2. Whether you're interested in creating smart contracts, dev tooling, protocol level development, AI image generation, or anything else - there are a lot of topics that have existed for decades that are still highly relevant and useful. A large part of what you would like to do today if you're reading this is learning how to build on top of ***web technologies*** - things that a user can access through a web browser.

In this lesson, I will go over a few key terminologies you will definitely come across as you begin your journey.

## Frontend Technologies?

**A frontend is the interface with which a user interacts.** On the web, the frontend refers to a website you can browse around, click on things, maybe even write stuff. Mobile apps and desktop apps are also valid examples of frontend interfaces.

For the purposes of this lesson, we're going to focus on web technologies.

Have you ever thought about how no matter what web browser you're using - Chrome, Firefox, Safari, Edge, etc - a website looks the exact same on all of them?

This happens because of **Web Standards.** An organization known as the **W3C (World Wide Web Consortium)** sets standards, a set of explicit rules, that all companies such as Google, Microsoft, Mozilla, Apple, etc. must follow when building things like web browsers across different devices and across different operating systems.

Particularly, these standards are set around the three key things every web developer needs to have some familiarity with - **HTML, CSS, and JavaScript**.

**HTML** is the language used to **visually place elements on your screen**. Things like this paragraph you're reading right now, things like buttons, things like a dropdown menu. You visually place elements across the screen using HTML.

**CSS** is a **styling language used to add styles** and your own custom flair to these elements. By default, HTML elements look boring and plain. Remember those old 90s websites? Yeah, that was plain HTML. CSS allows you to customize things about HTML elements. Such as making a button round instead of a rectangle, changing the font of a paragraph, having some **bold text** or <u>underlined text</u>, and so on.

And finally, to tie it all together comes JavaScript. **JavaScript** is arguably the most important aspect of building on web technologies. It is a fully functional programming language that is **used to add real functionality to your website**.

With HTML and CSS, you can place elements on a screen and make them look nice - but they won't actually *do* anything. Your button wouldn't actually do anything if you click it, more posts on Instagram will not load as you kept scrolling down, and so on.

JavaScript allows you to add real interactivity and functionality to your websites. It is, without a doubt, the language of the web. A website without JavaScript has no functionality other than letting you look at things a certain way.

# Backend Technologies?

**The backend refers to that part of a software that allows it to operate and cannot be directly accessed by the user.** Most private data, business logic, data processing, etc. happens on the backend, and the frontend is to provide a visual representation of that data.

For example, consider Instagram - Instagram has billions and billions of photos and videos on their platform. Some of those photos are posted by private accounts, photos that only people who follow that private account can see and anyone else cannot. The filtering of photos required for Instagram to show you your feed on the timeline vs. what they actually have available to them happens on their backend. It is important the backend is not directly accessible by users because otherwise private data can be leaked.

Backend services can be written in a **variety of programming languages** - **JavaScript, Rust, Go, Python, C#**, and many more. As you're starting your journey however, since you're going to be learning JavaScript anyway for the frontend, you might as well use JavaScript for the backend as well. While each programming language has its own pros and cons, they are not enough for a beginner to try to master multiple different languages and paradigms at once.